

dslar: l’ecosistema di Larsen autoregolato di Di Scipio

Diario di studio e ricostruzione per la suite SEAM-LTM

Giuseppe Silvi — SEAM

2026

Sommario

Questo documento è un diario di studio passo passo. Analizza la patch `LAR.pd` di Agostino Di Scipio a partire dal suo sorgente e dalla letteratura primaria dell’autore. Ne ricostruisce il motore di autoregolazione del feedback acustico, la legge di controllo omeostatica e la strategia di indipendenza dalla frequenza di campionamento, preparando le fasi successive: libreria Faust `sds`, esempio compilabile e plugin.

Indice

1	Introduzione e motivazione	2
2	Anatomia della patch	2
2.1	Il ramo audio	2
2.2	Il ramo di analisi	2
2.3	Verdetto sull’I/O: mono	2
3	Contesto e fonti	4
4	Analisi del segnale	4
4.1	Il ramo audio	5
4.2	Il ramo di analisi	5
4.3	Il bus <code>audioLAR</code> e il monitor	5
5	La legge di controllo omeostatica	5
5.1	Semantica di <code>env~</code> e <code>dbtorms</code>	5
5.2	La legge e i suoi limiti	6
5.3	Il ruolo dell’esponente 40	6
5.4	Un nuovo mattone <code>sds</code>	6
6	Compatibilità con la frequenza di campionamento	7
6.1	L’unica costante relativa ai campioni	7
6.2	Costanti già indipendenti da SR	7
6.3	Ritardi di decorrelazione e numero primo successivo	7
6.4	Piano di validazione	7
7	Mappa di decomposizione	7
7.1	Oggetti Pd da clonare e studio sul sorgente	8
8	Sintesi e ponte alla Fase 2	9
9	Fase 3 — completamento della specifica Faust	9

1 Introduzione e motivazione

LAR è la porta d'ingresso al capitolo che la suite SEAM-LTM dedica ad Agostino Di Scipio. La patch realizza un solo anello di retroazione acustica (Larsen) e una sola legge di controllo che ne regola l'ampiezza. Questa economia di mezzi la rende interamente leggibile in un colpo d'occhio. Un microfono, un altoparlante, un guadagno ricalcolato istante per istante dall'involuppo del segnale. Studiare LAR per prima fissa il vocabolario minimo (retroazione, involuppo, legge omeostatica, rampe di levigatura) che le patch successive di Di Scipio estendono verso ecosistemi con più nodi interagenti. Il diario parte da qui per capire un anello e una legge prima di affrontare le architetture più ampie.

2 Anatomia della patch

Questa sezione ricostruisce la struttura di `LAR.pd` leggendo direttamente i record `#X obj`, `#X msg` e `#X connect` del sorgente. Al centro della patch un moltiplicatore `*~` riceve il segnale del microfono e ne fissa il guadagno d'ingresso. Da questo nodo condiviso partono due rami. Il ramo audio trasporta il suono lungo l'anello di Larsen fino agli altoparlanti (`#X connect 11 0 33 0`). Il ramo di analisi misura lo stesso segnale e ne ricava il guadagno che l'anello deve assumere (`#X connect 11 0 30 0`).

2.1 Il ramo audio

Il segnale d'ingresso arriva da `adc~ 1`, un solo canale di microfono, e alimenta il moltiplicatore d'ingresso (`#X connect 9 0 11 0`). Quel moltiplicatore scala l'ingresso con una rampa `line` pilotata dal messaggio `$1 2000`, una dissolvenza di due secondi comandata da un interruttore `tgl`. Dal nodo d'ingresso il ramo audio attraversa il passa-alto `hip~ 100`, il pre-guadagno `*~ 1` selezionabile fra i valori 1, 2 e 4, e la linea di ritardo `tab1` di 50 ms. Il segnale ritardato entra nel moltiplicatore d'anello `*~`, dove viene scalato dal guadagno calcolato dal ramo di analisi (`#X connect 28 0 5 0` e `#X connect 7 0 5 1`). Due moltiplicatori d'uscita `*~` applicano l'interruttore di silenziamento `tgl` e portano il segnale a `dac~` e a `s~ audioLAR`.

2.2 Il ramo di analisi

Il ramo di analisi preleva lo stesso segnale d'ingresso attraverso la linea di ritardo `tab2` di 20 ms (`#X connect 11 0 30 0`). L'involuppo `env~ 2048` ne misura il livello su una finestra di 2048 campioni e lo restituisce in decibel (`#X connect 31 0 20 0`). La catena `dbtorms, - 1, abs` e `pow 40` trasforma quel livello nel fattore di guadagno dell'anello. Il valore risultante viene formattato dal messaggio `$1 200` e levigato da una rampa `line` di 200 ms prima di rientrare nel moltiplicatore d'anello (`#X connect 17 0 7 0`). Questa rampa chiude l'anello: l'ampiezza misurata all'ingresso determina il guadagno applicato all'uscita. La legge di controllo che governa questa trasformazione è analizzata nella sezione 5.

La patch contiene inoltre un sottopatch `pd input` con i filtri `hip~ 200` e `lop~ 15000`, che nel sorgente resta scollegato dal percorso del segnale.

2.3 Verdetto sull'I/O: mono

L'ingresso è mono: `adc~ 1` dichiara un solo canale e alimenta un unico moltiplicatore d'ingresso (`#X connect 9 0 11 0`). Anche l'uscita è mono, replicata sui due canali di `dac~`. I due moltiplicatori d'uscita ricevono la stessa coppia di sorgenti: il moltiplicatore d'anello `*~` (`#X connect 5 0 12 0` e `#X connect 5 0 13 0`) e l'interruttore `tgl` (`#X connect 10 0 12 1` e `#X connect`

Oggetto	Argomenti	Ruolo	Ramo
adc~	1	Ingresso microfono (mono)	Audio
*~	—	Guadagno d'ingresso	Audio
hip~	100	Passa-alto d'anello	Audio
*~	1	Pre-guadagno (1/2/4)	Audio
delwrite~	tab1 50	Scrittura ritardo d'anello	Audio
delread~	tab1 50	Lettura ritardo d'anello	Audio
*~	—	Moltiplicatore d'anello	Audio
*~	—	VCA d'uscita (L/R)	Audio
tgl	—	Silenziamento uscita	Audio
dac~	—	Uscita altoparlante	Audio
s~	audioLAR	Diramazione uscita	Audio
delwrite~	tab2 20	Scrittura tap di analisi	Analisi
delread~	tab2 20	Lettura tap di analisi	Analisi
env~	2048	Inviluppo del livello (dB)	Analisi
dbtorms	—	Conversione dB→RMS	Analisi
-	1	Scarto dal riferimento	Analisi
abs	—	Valore assoluto	Analisi
pow	40	Non linearità di controllo	Analisi
line	—	Rampa di controllo (200 ms)	Analisi
msg	\$1 200	Formattazione rampa di controllo	Analisi
line	—	Rampa d'ingresso (2000 ms)	Audio
msg	\$1 2000	Formattazione rampa d'ingresso	Audio
r~	audioLAR	Ripresa per monitor	Monitor
oscilloscope	—	Visualizzazione	Monitor
pd input	—	Pre-filtri scollegati	Dormiente

Tabella 1: Inventario degli oggetti di `LAR.pd`, con i due rami del segnale. Ogni valore è trascritto verbatim dal sorgente.

10 0 13 1). Il primo va al canale sinistro di `dac~` (`#X connect 12 0 0 0`), il secondo al canale destro (`#X connect 13 0 0 1`). I due canali calcolano quindi lo stesso prodotto a partire dalle stesse sorgenti, così l'uscita è un mono duplicato. Il verdetto è netto: `dslar` è un plugin mono, un ingresso e un'uscita. Per la porta in SEAM-LTM questo fissa la normalizzazione a un canale d'ingresso e un canale d'uscita, con l'eventuale duplicazione lasciata all'host.

3 Contesto e fonti

La poetica ecosistemica di Agostino Di Scipio tratta la sala come una risorsa e la retroazione come materiale sonoro. Un sistema di elaborazione entra in contatto con l'ambiente che lo ospita e ne trae l'energia per sostenersi, secondo i principi di “system / ambiente coupling” e “self-organisation” che l'autore espone in [2]. In questa prospettiva la risonanza della stanza e il rumore di fondo sono la “basic resource” da cui nasce qualcosa che chiamiamo musica [4]. L'ecosistema si compie quando la rete di interazioni include il proprio *oikos*, lo spazio risonante in cui il sistema trova le risorse per vivere [4]. LAR mette in scena esattamente questo ascolto: la stanza ascolta se stessa mentre suona, e il guadagno d'anello è la risposta del sistema a ciò che sente.

Le prime opere ecosistemiche di Di Scipio nascono come patch di segnale in KYMA5.2, come l'autore dichiara descrivendo l'Audible Eco-Systemic Interface [2]. L'opera realizzata interamente in Pure Data è *Modes of Interference*, il sistema di retroazione audio per tromba ed elettronica documentato in [3]. `LAR.pd` è una patch di Pure Data, e questo la colloca nella linea di *Modes of Interference*, in accordo con il nome delle patch sorelle (`LAR MOI1`).

Il testo del 2006 descrive la funzione della patch con parole che ricalcano la struttura di `LAR.pd`. Il compito è “implement a nonlinear coupling between a microphone and a pair of loudspeakers, such that a self-regulating feedback dynamics is established” ([3], abstract). Il guadagno d'anello è “an adaptive process, depending on the total sound level”, pensato per “keep the system in equilibrium at all times” ([3], p. 2, §2.2.2). L'obiettivo omeostatico è tenere il guadagno “high enough to result into Larsen tones and other artifacts, but not too high to cause clipping” ([3], p. 3).

Questa descrizione si sovrappone punto per punto all'anatomia ricostruita nella sezione 2. L'inviluppo `env~ 2048` misura il “total sound level” del segnale d'anello. La catena `dbtorms, -1, abs` e `pow 40` realizza l’“adaptive process” che scala il guadagno in senso inverso all'ampiezza misurata. La coppia `s~ audioLAR` e `r~ audioLAR` chiude l'anello attraverso la stanza, che diventa parte integrante del percorso del segnale.

`LAR.pd` isola il cuore autoregolante di *Modes of Interference* dagli strati di trasformazione granulare (le sotto-patch `pd gr8`, `pd gr4`, `pd gr2`) che il testo del 2006 presenta come estensioni del “fundamental audio feedback loop” [3]. Restano l'anello fondamentale e la sola legge di controllo, la coppia minima che questo studio porta in SEAM-LTM.

La stessa idea del Larsen acustico come sorgente e materiale attraversa il ciclo Audible Eco-systemics, in particolare *Feedback Study*, dove la retroazione tra microfoni e altoparlanti è l'unico generatore sonoro [1]. LAR condivide con quelle opere l'omeostasi del livello di stanza, e la traduce nella forma più essenziale di un solo anello e di una sola rampa di controllo.

4 Analisi del segnale

Questa sezione segue il segnale campione per campione lungo i due rami che escono dal nodo d'ingresso condiviso. Il ramo audio porta il suono attraverso l'anello di Larsen; il ramo di analisi misura lo stesso suono e ne ricava il guadagno che l'anello deve assumere.

4.1 Il ramo audio

Il segnale nasce da `adc~ 1`, un solo canale di microfono. Entra nel moltiplicatore d'ingresso, dove una dissolvenza in salita di 2s (rampa `line` pilotata dal messaggio `$1 2000`) porta il guadagno d'ingresso a regime senza transienti. Il passa-alto `hip~ 100` rimuove la componente continua e il rombo di sala sotto 100 Hz. Il pre-guadagno `*~ 1`, commutabile fra 1, 2 e 4, fissa l'energia che alimenta l'anello. La linea di ritardo `tab1` di 50 ms (`delwrite~/delread~`) impone lo sfasamento temporale che innesca il Larsen. Il segnale ritardato entra nel moltiplicatore d'anello, dove viene scalato dal guadagno omeostatico prodotto dal ramo di analisi. I due moltiplicatori d'uscita applicano l'interruttore di silenziamento `tgl` e inviano il segnale a `dac~` e a `s~ audioLAR`.

4.2 Il ramo di analisi

Il ramo di analisi preleva lo stesso segnale d'ingresso attraverso la linea di ritardo `tab2` di 20 ms, che lo decorrela dal ramo audio prima della misura. L'inviluppo `env~ 2048` calcola il livello RMS su una finestra di 2048 campioni e lo restituisce in decibel secondo la convenzione di Pd. La catena di controllo `dbtorms`, `- 1`, `abs` e `pow 40` converte quel livello nel fattore di guadagno dell'anello. Il valore risultante viene formattato dal messaggio `$1 200` e levigato da una rampa `line` di 200 ms, che chiude l'anello riportando il guadagno al moltiplicatore d'anello. La legge di controllo che governa questa catena è derivata nella sezione 5.

4.3 Il bus audioLAR e il monitor

La coppia `s~ audioLAR` e `r~ audioLAR` costituisce un anello acustico reale, oltre il semplice instradamento interno. L'uscita di `dslar` passa per gli altoparlanti, attraversa la stanza, rientra dal microfono e ritorna al nodo d'ingresso: il bus `audioLAR` è la traccia software di questo anello acustico che si chiude nell'*oikos*, lo spazio risonante che alimenta il sistema. Il monitor `r~ audioLAR → oscilloscope` osserva la forma d'onda dell'anello lasciando il segnale inalterato, ed è lo strumento con cui l'autore verifica a occhio la “breathing” dell'ecosistema.

La Tabella 2 riassume i due percorsi come catene di blocchi in serie.

Ramo	Catena del segnale
Audio	<code>adc~ 1 → dissolvenza (2s) → hip~ 100 → pre-guadagno (1/2/4) → tab1 (50 ms) → moltiplicatore d'anello → VCA → dac~, s~ audioLAR</code>
Analisi	<code>nodo d'ingresso → tab2 (20 ms) → env~ 2048 → dbtorms → - 1 → abs → pow 40 → line (200 ms) → moltiplicatore d'anello</code>
Monitor	<code>r~ audioLAR → oscilloscope</code>

Tabella 2: I due rami di `LAR.pd` come catene di blocchi in serie, più il ramo di monitoraggio. Il moltiplicatore d'anello è il punto in cui il ramo di analisi agisce sul ramo audio.

5 La legge di controllo omeostatica

Il cuore autoregolante di `dslar` è la catena `env~ 2048 → dbtorms → - 1 → abs → pow 40`. Per derivarla correttamente occorre fissare la semantica esatta degli oggetti di Pd coinvolti, evitando di trattare `env~` come un generico “inviluppo”.

5.1 Semantica di `env~` e `dbtorms`

L'oggetto `env~ N` emette il livello RMS del suo ingresso in decibel, secondo la convenzione di Pd per cui 100 dB corrispondono a un'ampiezza RMS pari a 1.0:

$$\text{env}(x) = 100 + 20 \log_{10}(\text{rms}(x)), \quad N = 2048 \text{ campioni.} \quad (1)$$

L'oggetto `dbtorms` è l'inverso esatto di questa convenzione:

$$\text{dbtorms}(\text{dB}) = 10^{(\text{dB}-100)/20}. \quad (2)$$

Componendo i due si recupera l'ampiezza RMS lineare del segnale d'anello, che indichiamo con r :

$$r = \text{dbtorms}(\text{env}(x)) = \text{rms}(x) \in [0, 1] \text{ (circa)}. \quad (3)$$

La legge di controllo va quindi ragionata sull'ampiezza RMS *lineare* r , non sui decibel.

Verifica sul sorgente. Questo comportamento è confermato leggendo il sorgente di Pure Data. L'oggetto `env~` (classe `sigenv` in `src/d_ctl.c`) calcola la media dei quadrati del segnale pesata da una finestra di Hann normalizzata su N campioni, e ne emette `powtodb`. La funzione `powtodb` (in `src/x_acoustics.c`) vale $100 + 10 \log_{10}(\text{potenza})$, così l'uscita di `env~` esprime la potenza in decibel con 100 dB pari a potenza unitaria. La funzione `dbtorms` vale $10^{(\text{dB}-100)/20}$, e composta con `env~` restituisce $\langle x^2 \rangle^{1/2}$, cioè l'RMS esatto. La riscrittura in Faust e in C++ riproduce perciò la media dei quadrati pesata Hann seguita da `powtodb`, e conserva lo stesso comportamento a ogni frequenza di campionamento. Questa annotazione è la traccia del controllo eseguito sul codice aperto, e sostiene una scrittura continua dal patch al plugin.

5.2 La legge e i suoi limiti

Applicando `- 1`, `abs` e `pow 40` all'ampiezza lineare r si ottiene il guadagno d'anello:

$$g(t) = |r(t) - 1|^{40}, \quad r(t) = \text{dbtorms}(\text{env}(t)). \quad (4)$$

I due limiti di (4) descrivono l'omeostasi. Stanza silenziosa: $r \rightarrow 0$, quindi $|r - 1| \rightarrow 1$ e $g \rightarrow 1$, il guadagno si apre e l'anello riprende a suonare. Stanza satura: $r \rightarrow 1$, quindi $|r - 1| \rightarrow 0$ e $g \rightarrow 0$, il guadagno si chiude e l'anello si spegne prima del clipping. Il sistema insegue perciò un equilibrio: ogni volta che l'anello cresce, il guadagno si riduce; ogni volta che tace, il guadagno risale.

5.3 Il ruolo dell'esponente 40

L'esponente 40 trasforma la mappa $|r - 1|$ da rampa dolce in curva quasi a soglia. Sull'intervallo $r \in [0, 1]$ vale $|r - 1| = 1 - r$, e questo termine resta vicino a 1 solo per valori piccoli di r : elevato alla quarantesima potenza, precipita non appena r supera circa 0.1. Con $r = 0.11$ si ha $(1 - r)^{40} = 0.89^{40} \approx 10^{-2}$, un guadagno già ridotto a un centesimo; con $r = 0.9$ il guadagno è $0.1^{40} \approx 10^{-40}$, di fatto nullo. L'anello si autoregola perciò a un livello RMS basso: il punto di equilibrio operativo si colloca all'incirca in $r \in [0, 0.12]$, un regime di Larsen a bassa ampiezza, ben lontano dalla saturazione. L'esponente 40 chiude il guadagno non appena r supera circa 0.1: il crollo avviene a basso livello di stanza, non a ridosso della saturazione. La sensibilità della legge, $dg/dr = -40(1 - r)^{39}$, raggiunge il modulo massimo (40) proprio per $r \rightarrow 0$, cioè nel punto di equilibrio: questa risposta ripida, più che la semplice sottrazione dal riferimento, produce il “respiro” e il pulsare che Di Scipio descrive come “self-regulating feedback”.

5.4 Un nuovo mattone `sds`

Questo idioma, ampiezza lineare $\rightarrow$ scarto dal riferimento $\rightarrow$ valore assoluto $\rightarrow$ potenza elevata, è nuovo rispetto alla famiglia `sds.map*` esistente delle librerie SEAM. La legge (4) diventa perciò un nuovo mattone `sds` da introdurre nella Fase 2, con l'esponente e il riferimento come parametri, insieme alla conversione `env~/dbtorms` che la precede.

6 Compatibilità con la frequenza di campionamento

Di Scipio scrive le sue patch alla frequenza di riferimento $SR_{\text{ref}} = 44\,100$ Hz, mentre la suite SEAM-LTM lavora a frequenza arbitraria. Un porting corretto deve mantenere *invariante* il comportamento temporale al variare di SR.

6.1 L'unica costante relativa ai campioni

In tutta `LAR.pd` la sola costante espressa in campioni è la finestra di `env~ 2048`. Ancorata nel tempo alla frequenza di riferimento, essa vale

$$T_{\text{env}} = \frac{2048}{44100} \text{ s} \approx 46.4 \text{ ms}, \quad N(\text{SR}) = \text{ba.sec2samp}(T_{\text{env}}) = \text{round}(T_{\text{env}} \cdot \text{SR}).$$

Alla frequenza di riferimento $N(44100) = 2048$, mentre a 96 kHz si ottiene $N \approx 4458$ campioni, che è ancora una finestra di 46.4 ms. Mantenere il letterale `2048` a 96 kHz ridurrebbe la finestra a circa 21 ms, accelerando la misura RMS e alterando il “respiro” dell’ecosistema. Ancorare la finestra nel tempo e ricostruire i campioni alla frequenza effettiva mantiene invece il comportamento identico a ogni SR.

6.2 Costanti già indipendenti da SR

Le linee di ritardo `delread~/delwrite~ (tab1 50 ms, tab2 20 ms)` e le rampe `line (2000 ms e 200 ms)` hanno argomenti già in millisecondi, quindi sono intrinsecamente indipendenti da SR. Anche il passa-alto `hip~ 100` è espresso in hertz e resta invariante, a patto che i coefficienti del filtro siano ridisegnati alla frequenza effettiva.

6.3 Ritardi di decorrelazione e numero primo successivo

I ritardi di decorrelazione da 20 ms e 50 ms, una volta convertiti in campioni alla frequenza effettiva, possono essere agganciati al numero primo successivo. La funzione esiste già: `sff.np` in Faust e il `nextPrime` scritto a mano di `ddelay` in C++. Snappare i ritardi a lunghezze prime rende coprimi i ritardi di più istanze di `dsLAR`, così che rinforzino risonanze a pettine diverse e restino decorrelate tra loro. Resta una scelta caso per caso, coerente con la convenzione di porting di Di Scipio.

6.4 Piano di validazione

La strategia si verifica con un test A/B comportamentale fra 44.1 kHz e 96 kHz. Con la finestra ancorata nel tempo, le due frequenze devono produrre lo stesso involuppo nel dominio del tempo e lo stesso periodo di “respiro” dell’anello; una divergenza segnala il rientro del letterale `2048` nel percorso di calcolo.

Un secondo test A/B confronta la patch originale in Pure Data con l’implementazione in Faust, a parità di ingresso. Pure Data elabora a blocchi di 64 campioni: l’involuppo aggiorna al suo `period`, la rampa `line` aggiorna a `control-rate`, e l’anello di feedback porta un ritardo di un blocco, mentre Faust lavora per campione. Questo controllo verifica che il salto al calcolo per-campione conservi il comportamento dell’anello, in particolare la temporizzazione del feedback e il ritmo di aggiornamento del segnale di controllo. Una divergenza sostanziale indica quali oggetti Pd il porting deve emulare a `control-rate` o con il ritardo di un blocco per restare fedele.

7 Mappa di decomposizione

La Tabella 3 smista ogni blocco di `LAR.pd` verso una delle quattro destinazioni previste dalla specifica di porting. Un blocco diventa un clone `pdclone` quando replica un oggetto di Pure

Data la cui semantica esatta differisce dagli standard Faust, come `env~`: questi cloni vivono nella libreria `seam.pdclone.lib`, come `seam.cyclone.lib` raccoglie gli oggetti di Max. Un blocco diventa un nuovo mattone `sds` quando incarna un idiomma di Di Scipio ancora assente dalla libreria, e compone i cloni `pdclone` invece di re-implementarli. Un blocco resta circuito locale di `dslar` quando è specifico di questa patch, in attesa che una seconda patch lo rivendichi. Un blocco si copre con uno standard Faust già disponibile quando realizza una funzione generica, come una rampa o un filtro.

Blocco patch	Destino	Nota
<code>env~ 2048 (RMS→dB)</code>	clone <code>pdclone</code>	Oggetto Pd <code>env~</code> da clonare come <code>pdclone.env</code> (media dei quadrati pesata Hann + <code>powtdb</code>); nessun equivalente pronto in Faust; <code>sds</code> lo compone
<code>dbtorms → - 1 → abs → pow 40</code>	mattone <code>sds</code>	Legge omeostatica; idioma nuovo, cfr. sezione 5
Dissolvenza \$1 2000→line; rampa di controllo \$1 200→line	Faust std	Levigatura; già coperta dagli standard Faust
<code>hip~ 100</code>	Faust std	Passa-alto; coefficienti ridisegnati alla frequenza effettiva
<code>delwrite~/delread~ tab1 50, tab2 20</code>	circuito LAR	Ritardi di decorrelazione; next-prime opzionale (<code>sff.np/ddelay</code>)
Pre-guadagno 1/2/4, VCA finale	circuito LAR	Diventano parametri esposti dal plugin
<code>s~/r~ audioLAR</code>	I/O del plugin	L'anello si chiude fuori dal plugin, nella stanza

Tabella 3: Mappa di decomposizione di `LAR.pd`: destinazione di ogni blocco fra clone `pdclone` di un oggetto Pd, nuovo mattone `sds`, circuito locale di `dslar` e standard Faust esistente.

7.1 Oggetti Pd da clonare e studio sul sorgente

I blocchi diretti in `seam.pdclone.lib` si replicano fedelmente leggendo il sorgente di Pure Data. La Tabella 4 elenca gli oggetti coinvolti e lo stato della verifica sul codice.

Oggetto	Sorgente Pd	Esito della verifica
<code>env~</code>	<code>d_ctl.c</code> (<code>sigenv</code>)	Media dei quadrati pesata Hann su N campioni, uscita <code>powtdb</code> ; nessun equivalente Faust pronto
<code>dbtorms</code> , <code>powtdb</code>	<code>x_acoustics.c</code>	Riferimento a 100 dB: $\text{powtdb} = 100 + 10 \log_{10}(\text{pot})$, $\text{dbtorms} = 10^{(\text{dB}-100)/20}$
<code>hip~</code>	<code>d_filter.c</code> (<code>sighip</code>)	Passa-alto a un polo e uno zero: $\text{coef} = 1 - 2\pi f/\text{SR}$ (limitato a $[0, 1]$), $y = \frac{1+\text{coef}}{2}(w - w_{-1})$ con $w = x + \text{coef} w_{-1}$; diverso dal Butterworth di <code>fi.highpass</code> . Si realizza con elementi minimi Faust: <code>fi.pole(coef) : fi.zero(1) : *((1+coef)/2)</code>
<code>line</code>	<code>x_time.c</code>	Da studiare: rampa a control-rate, legata al kernel a blocchi (sezione 6)
<code>delread~</code> , <code>delwrite~</code>	<code>d_delay.c</code>	Da confermare rispetto a <code>de.delay</code>

Tabella 4: Oggetti di Pure Data che `dslar` replica, con la fonte nel codice di Pd e lo stato della verifica. Gli oggetti aritmetici (`*~`, `-`, `abs`, `pow`) coincidono con gli standard Faust e non richiedono un clone.

Il confine fra plugin e stanza merita una nota a parte. Nella patch il bus `audioLAR` sembra un instradamento interno, ma fisicamente il segnale esce da `dac~`, eccita il Larsen acustico e rientra

da `adc~`. `dslar` è quindi un plugin realmente ingresso→uscita: l’anello di autoregolazione si chiude nell’aria della stanza, il vero percorso del segnale. Questo vincolo va documentato come condizione d’uso, insieme all’accortezza di posizionamento fra microfono e altoparlante che il Larsen richiede. Per permettere lo studio dell’ecosistema anche in cuffia, il plugin espone un anello interno opzionale, un sostituto digitale del Larsen acustico. Un interruttore nella GUI richiude il segnale dal ramo audio al ramo di analisi dentro al plugin stesso, sostituendo il percorso acustico con un percorso digitale equivalente.

8 Sintesi e ponte alla Fase 2

Questo studio prepara il terreno per la Fase 2 del programma di porting Di Scipio → SEAM. La mappa di decomposizione della sezione 7 fissa sette parametri da esporre nell’interfaccia di `dslar`. Il drive d’ingresso regola il pre-guadagno prima dell’anello. Il target fissa il riferimento di equilibrio della legge di controllo, il valore `-1` dell’equazione (4). La steepness regola la pendenza della legge, l’esponente `40` della stessa equazione. Il tempo di levigatura del controllo corrisponde alla rampa di `200 ms` che chiude l’anello. Il tempo di decorrelazione corrisponde ai ritardi di `20 ms` e `50 ms` dei due rami. L’uscita regola il livello finale verso l’host. L’anello interno, attivabile o disattivabile, abilita lo studio in cuffia descritto nella sezione 7. La Fase 2 produce due oggetti, distribuiti su due librerie. Il primo è il clone `pdclone.env`, che replica `env~2048` come media dei quadrati pesata Hann seguita da `powtodb`, poiché gli standard Faust non offrono un equivalente pronto. Questo clone vive nella nuova libreria `seam.pdclone.lib`, che raccoglie gli oggetti di Pure Data replicati in Faust, come `seam.cyclone.lib` fa con gli oggetti di Max. Il secondo è la legge omeostatica dell’equazione (4), un mattone `sds` con riferimento ed esponente come parametri espliciti, che compone `pdclone.env` al suo ingresso. Il follower resta distinto da `sds.integrator(s)` che AE2 (`fc2003dsaae2`) già usa, una media rettangolare del valore assoluto con costante di tempo in secondi. Il mattone `sds` della legge omeostatica resta compatibile con le firme che AE2 già usa, a tutela del codice della composizione esistente. Restano aperte due domande per la Fase 2. Il nome e la firma esatti dei due oggetti si definiscono confrontandoli con l’uso reale in AE2. Se il ritardo di decorrelazione debba diventare un terzo mattone `sds` si decide solo quando una seconda patch lo rivendica, in coerenza con la politica di estrazione incrementale della libreria `sds`.

9 Fase 3 — completamento della specifica Faust

La lettura connessione-per-connessione di `LAR.pd` corregge l’inquadramento dell’anello. Il patch non ha retroazione interna: l’unica sorgente è `adc~`, e `r~ audioLAR` raggiunge solo l’oscilloscopio. La linea `tab1` di `50 ms` è quindi un ritardo *feedforward*, non una ricorsione: l’anello di Larsen è acustico (`dac~` → stanza → microfono → `adc~`), esterno al plugin, l’*oikos* che alimenta il sistema. Di conseguenza `dslar` è una catena mono feedforward pura, senza `~` di retroazione.

Il clone `spd.line` riproduce fedelmente il `line control-rate` di Pure Data. La rampa lineare segue la formula di `line_tick` ($v = setval + \min(elapsed/R, 1)(target - setval)$, con ripartenza dal valore corrente) ed è campionata al passo di grain di `20 ms` in una scala control-rate. La verifica contro un oracolo Python della formula Pd dà scarto massimo dell’ordine di 1×10^{-8} , sia sul gradino sia sul cambio di target a metà rampa. La differenza di fase dei tick (Pd allinea al messaggio, Faust al campione zero) è il residuo documentato, rinviato alla validazione block-kernel.

Ogni oggetto Pd replicato porta ora una doppia fonte: il sorgente C e l’help patch di Miller Puckette. Gli help patch sono copiati nel repository sotto `doc/references/pd-help` per non dipendere dal checkout esterno di Pure Data. La libreria `seam.discipio.lib` è stata rimessa in `src` e registrata in `seam.lib`: è in cura ed è il momento giusto per riportarla al suo posto. Il

mattone `sds.lar` assembla i pezzi verificati nel processore feedforward, e `dslar.dsp` lo espone con i sette parametri più il gate.

Questa documentazione è preparata per essere condivisa con Agostino Di Scipio, a validazione dell'intero processo di porting.

10 Fase 4 — il plugin C++

La specifica Faust congelata (`sds.lar`) diventa il contratto del porting a mano in C++, secondo la regola del progetto: Faust è la specifica, il C++ leggibile è il deliverable. Il cuore `dslar_dsp.h` è header-only e privo di SDK, quindi verificabile a sé con `doctest` contro gli stessi numeri d'oracolo della Fase 2–3.

Ogni oggetto Pd è portato come una piccola unità testata: i convertitori `powtodb/dbtorms` (`x_acoustics.c`), il passa-alto a un polo `hip~` (`d_filter.c`, topologia esatta con il π letterale 3.14159), la rampa control-rate `line` (`x_time.c`, scala a gradini di 20 ms), e l'involuppo RMS pesato Hann `env~` (`d_ctl.c`). L'`env` è reso come somma Hann su ring buffer calcolata alla cattura ogni `period` e tenuta tra le catture: stesso costo ammortizzato dell'overlap-add ma più leggibile, e float-identico perché la finestra Hann normalizzata somma a uno. L'indipendenza dalla frequenza di campionamento vive nel deliverable: la finestra è dimensionata `round(fs*2048/44100)` in `prepare(fs)`, così il comportamento temporale resta invariante col variare di f_s .

L'assemblaggio `Larsen` è una catena feedforward pura, senza retroazione interna, perché l'anello di Larsen è acustico ed esterno. La verifica end-to-end mostra che una continua a 0.5 porta il guadagno d'anello g a 0.5^{40} a 44.1 kHz, 48 kHz, 96 kHz e 192 kHz, confermando insieme l'omeostato e l'indipendenza da f_s . I contatori a control-rate della rampa e dell'involuppo avvolgono la loro fase invece di crescere senza limite, così il comportamento resta identico ma privo di overflow anche nelle esecuzioni prolungate di molte ore, native del contesto sostenuto di questa musica.

Il plugin nasce col sistema di metering condiviso della suite (`_common/seam_meter.h`), di cui `dslar` è il primo consumatore. Due barre in sola lettura mostrano r (RMS Hann, ingresso dell'omeostato) e g (guadagno d'anello all'uscita del `line`, fedele allo slider di Di Scipio), trasportate col meccanismo dei parametri read-only e normalizzate in dB a fondo scala -60 dBFS. Il plugin è un VST3 mono a un ingresso e un'uscita con otto parametri (potenza, drive, riferimento, ripidità, smoothing di controllo, ritardo d'anello, decorrelazione, uscita), e supera senza errori il validator dell'SDK. Il meter g rende visibile la controintuitività della legge: è pieno nel silenzio ($g = |0 - 1|^{40} = 1$, l'anello spinge verso il Larsen) e collassa verso lo zero all'equilibrio, dove il sistema si autoattenua. La resa ricca (una vista custom con scala in dB e la curva di trasferimento $g = |r - 1|^k$ col punto di lavoro vivo) è annotata come lavoro successivo.

Riferimenti bibliografici

- [1] Agostino Di Scipio. Audible Ecosystemics n.2a / Feedback Study (2003); n.2b / Feedback Study, Sound Installation (2004). Performance instructions, composer's document, 2003.
- [2] Agostino Di Scipio. 'Sound is the Interface': From Interactive to Ecosystemic Signal Processing. *Organised Sound*, 8(3):269–277, 2003.
- [3] Agostino Di Scipio. Using PD for Live Interactions in Sound: An Exploratory Approach. In *Proceedings of the 4th International Linux Audio Conference (LAC06)*, ZKM, Karlsruhe, 2006.
- [4] Agostino Di Scipio. Listening to Yourself through the Otherself: On Background Noise Study and Other Works. *Organised Sound*, 16(2):97–108, 2011.